LAB MANUAL

Subject: CA-8.7(C) Lab on Current Computing Trends (Big Data)

Class: MCA (Integrated)-IV

Semester: VIII

Assignment No. 1

Installing and Setting Up Hadoop in Pseudo-Distributed Mode in Windows 10

- To Perform setting up and installing Hadoop in the pseudo-distributed mode in Windows 10 using the following steps given below as follows. Let's discuss one by one.

Step 1: Download Binary Package
Download the latest binary from the following site as follows.

https://hadoop.apache.org/releases.html

Download

Hadoop is released as source code tarballs with corresponding binary tarballs for convenience. The downloads are distributed via mirror sites and should be checked for tampering using GPG or SHA-512.

Version	Release date	Source download	Binary download	Release notes
2.10.1	2020 Sep 21	source (checksum signature)	binary (checksum signature)	Announcement
3.1.4	2020 Aug 3	source (checksum signature)	binary (checksum signature)	Announcement
3.3.0	2020 Jul 14	source (checksum signature)	binary (checksum signature) binary-aarch64 (checksum signature)	Announcement
3.2.1	2019 Sep 22	source (checksum signature)	binary (checksum signature)	Announcement
2.9.2	2018 Nov 19	source (checksum signature)	binary (checksum signature)	Announcement

For reference, you can check the file save to the folder as follows.

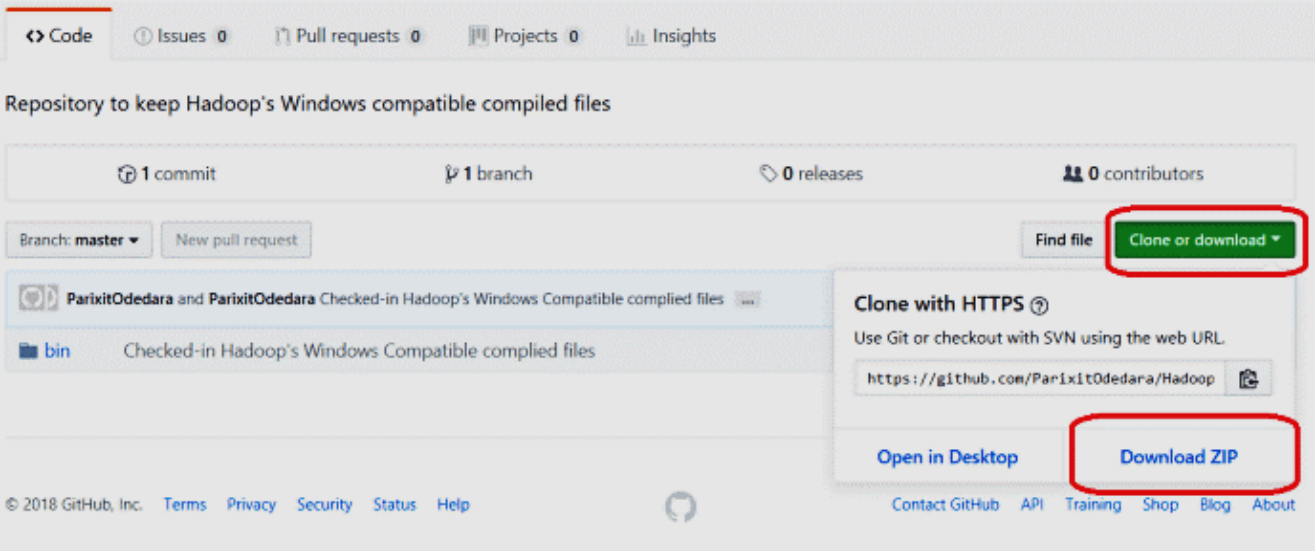
C:\BigData

Step 2: Unzip the binary package
Open Git Bash, and change directory (cd) to the folder where you save the binary package and then unzip as follows.

$ cd C:\BigData
MINGW64: C:\BigData
$ tar -xvzf hadoop-3.1.2.tar.gz

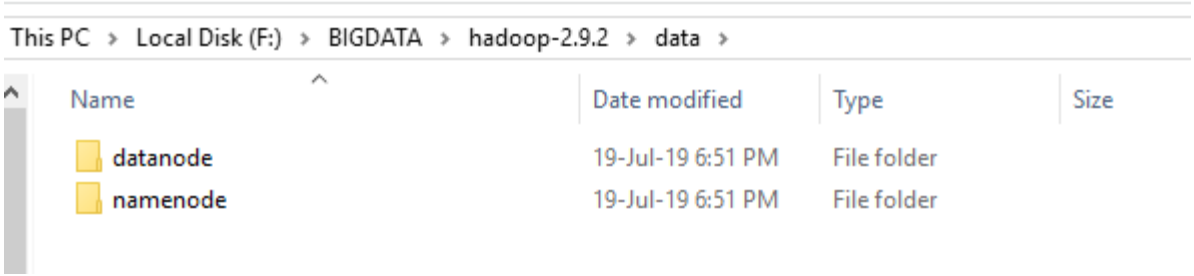
For my situation, the Hadoop twofold is extricated to C:\BigData\hadoop-3.1.2.

Next, go to this GitHub Repo and download the receptacle organizer as a speed as demonstrated as follows. Concentrate the compress and duplicate all the documents present under the receptacle envelope to C:\BigData\hadoop-3.1.2\bin. Supplant the current records too.



Step 3: Create folders for datanode and namenode

- Goto C:/BigData/hadoop-3.1.2 and make an organizer 'information'. Inside the 'information' envelope make two organizers 'datanode' and 'namenode'. Your documents on HDFS will dwell under the datanode envelope.



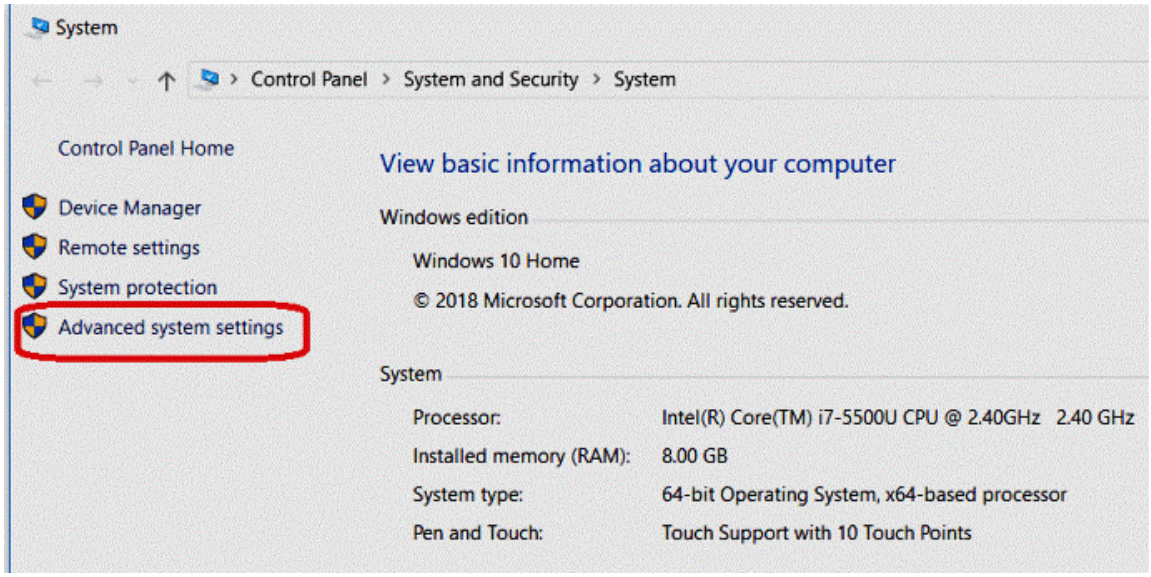
- Set Hadoop Environment Variables
- Hadoop requires the following environment variables to be set.

HADOOP_HOME=" C:\BigData\hadoop-3.1.2"
HADOOP_BIN="C:\BigData\hadoop-3.1.2\bin"
JAVA_HOME=<Root of your JDK installation>"

- To set these variables, navigate to My Computer or This PC.

Right-click -> Properties -> Advanced System settings -> Environment variables.

- Click New to create a new environment variable.

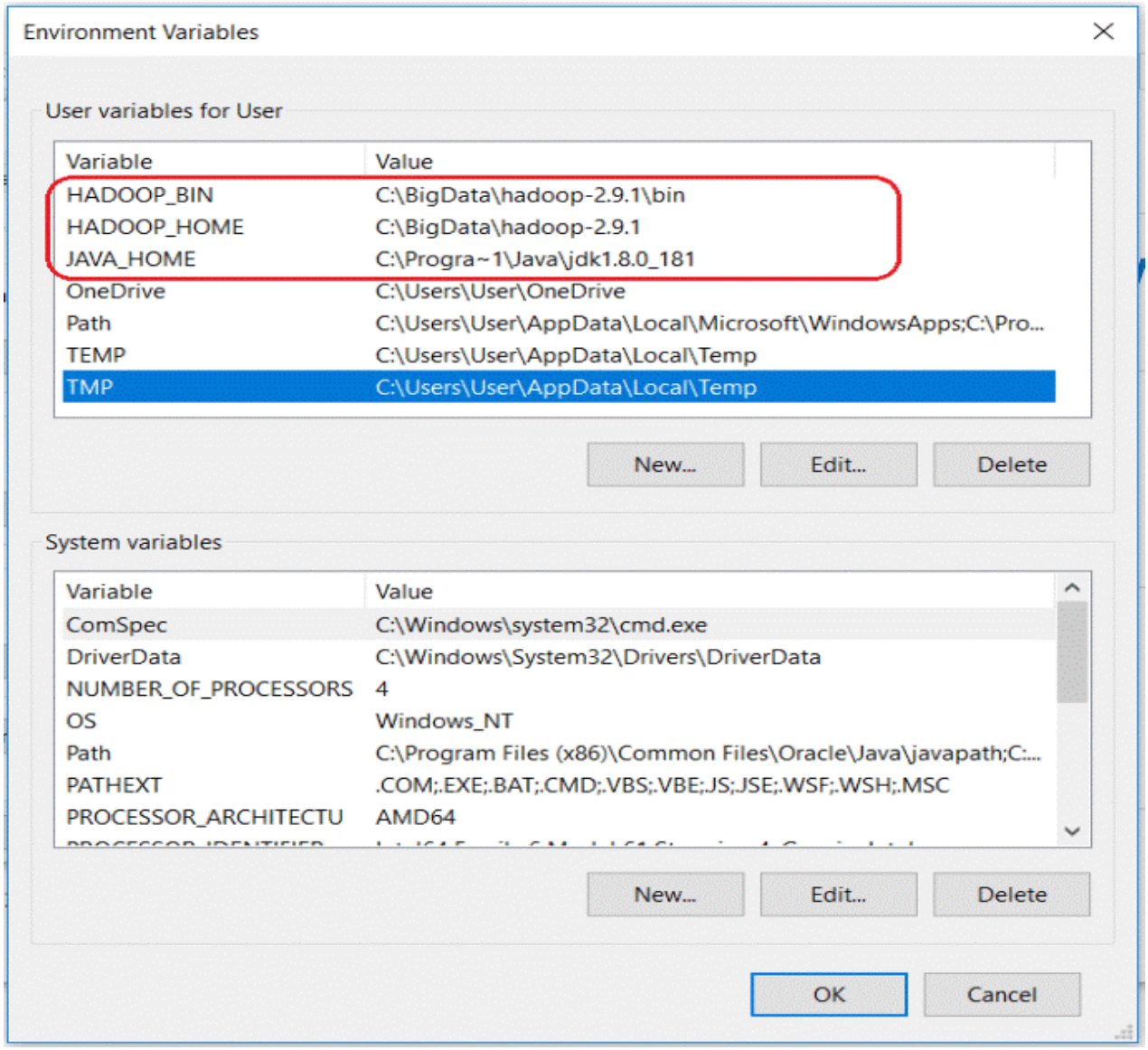


- In the event that you don't have JAVA 1.8 introduced, at that point you'll have to download and introduce it first. In the event that the JAVA_HOME climate variable is now set, at that point check whether the way has any spaces in it (ex: C:\Program

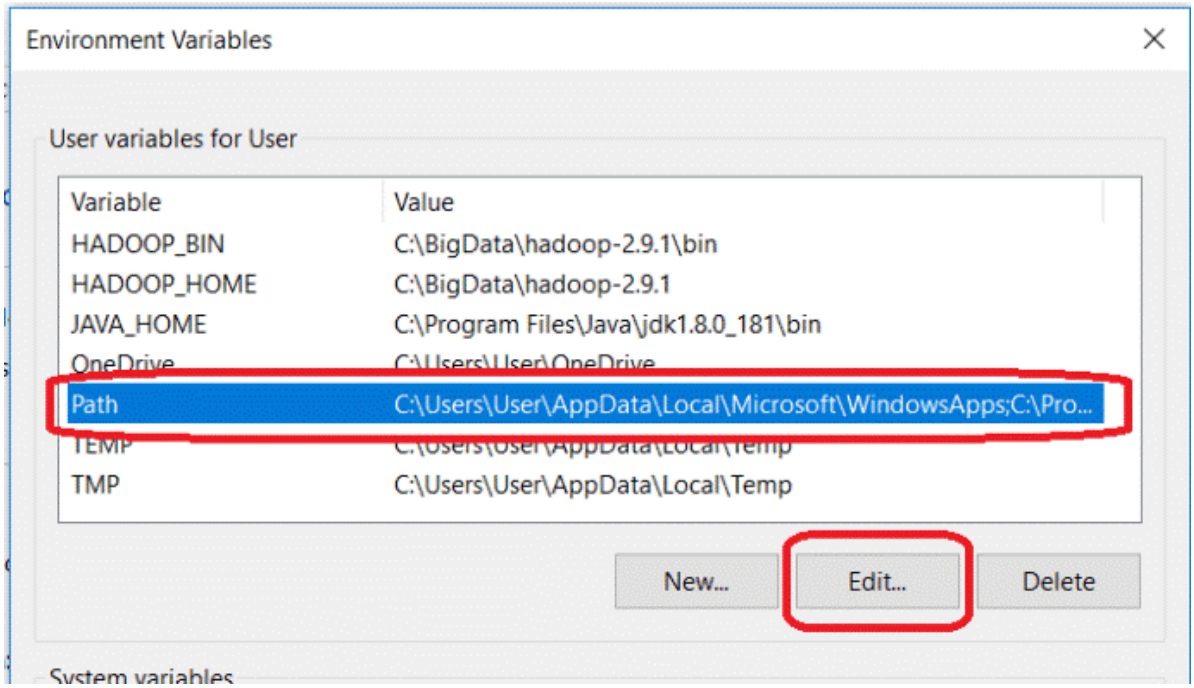
Files\Java\…). Spaces in the JAVA_HOME way will lead you to issues. There is a stunt to get around it. Supplant 'Program Files 'to 'Progra~1'in the variable worth. Guarantee that the variant of Java is 1.8 and JAVA_HOME is highlighting JDK 1.8.

Step 4: To make Short Name of Java Home path

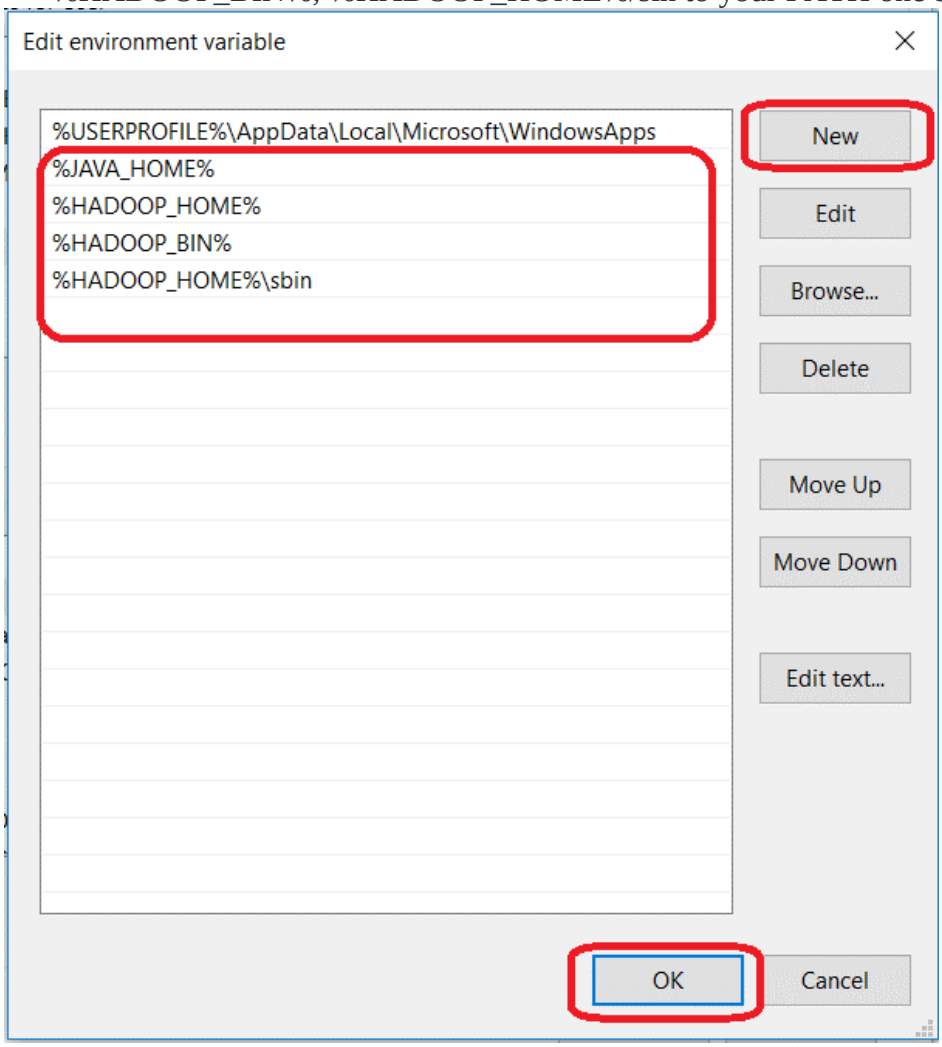
```
C:\Program Files\Java\jdk1.8.0_111>for %I in (.) do echo %~sI

C:\Program Files\Java\jdk1.8.0_111>echo C:\PROGRA~1\Java\JDK18~1.0_1
C:\PROGRA~1\Java\JDK18~1.0_1
```



- Set Hadoop Environment Variables
- Edit PATH Environment Variable



- Click on New and Add %JAVA_HOME%, %HADOOP_HOME%, %HADOOP_BIN%, %HADOOP_HOME%/sin to your PATH one by one.



- Now we have set the environment variables, we need to validate them. Open a new Windows Command prompt and run an echo command on each variable to confirm they are assigned the desired values.

echo %HADOOP_HOME%
echo %HADOOP_BIN%
echo %PATH%

- On the off chance that the factors are not instated yet, at that point it can likely be on the grounds that you are trying them in an old meeting. Ensure you have opened another order brief to test them.

Step 5: Configure Hadoop
Once environment variables are set up, we need to configure Hadoop by editing the following configuration files.

hadoop-env.cmd
core-site.xml
hdfs-site.xml
mapred-site.xml
yarn-site.xml
hadoop-env.cmd

First, let's configure the Hadoop environment file. Open C:\BigData\hadoop-3.1.2\etc\hadoop\hadoop-env.cmd and add below content at the bottom

```
set HADOOP_PREFIX=%HADOOP_HOME%
set HADOOP_CONF_DIR=%HADOOP_PREFIX%\etc\hadoop
set YARN_CONF_DIR=%HADOOP_CONF_DIR%
set PATH=%PATH%;%HADOOP_PREFIX%\bin
```

Step 6: Edit hdfs-site.xml
After editing core-site.xml, you need to set the replication factor and the location of namenode and datanodes. Open C:\BigData\hadoop-3.1.2\etc\hadoop\hdfs-site.xml and below content within <configuration> </configuration> tags.

```
<configuration>
<property>
<name>dfs.replication</name>
<value>1</value>
</property>
<property>
<name>dfs.namenode.name.dir</name>
<value>C:\BigData\hadoop-3.2.1\data\namenode</value>
</property>
<property>
<name>dfs.datanode.data.dir</name>
<value>C:\BigData\hadoop-3.1.2\data\datanode</value>
</property>
</configuration>
```

Step 7: Edit core-site.xml
Now, configure Hadoop Core's settings. Open C:\BigData\hadoop-3.1.2\etc\hadoop\core-site.xml and below content within <configuration> </configuration> tags.

```
<configuration>
<property>
<name>fs.default.name</name>
<value>hdfs://0.0.0.0:19000</value>
</property>
</configuration>
```

Step 8: YARN configurations
Edit file yarn-site.xml
Make sure the following entries are existing as follows.

<configuration> <property>
<name>yarn.nodemanager.aux-services</name>
<value>mapreduce_shuffle</value> </property>
<property>
<name>yarn.nodemanager.aux-services.mapreduce.shuffle.class</name>
<value>org.apache.hadoop.mapred.ShuffleHandler</value>
</property>
</configuration>

Step 9: Edit mapred-site.xml

At last, how about we arrange properties for the Map-Reduce system. Open C:\BigData\hadoop-3.1.2\etc\hadoop\mapred-site.xml and beneath content inside <configuration> </configuration> labels. In the event that you don't see mapred-site.xml, at that point open mapred-site.xml.template record and rename it to mapred-site.xml

<configuration>
<property>
<name>mapreduce.job.user.name</name> <value>%USERNAME%</value>
</property>
<property>
<name>mapreduce.framework.name</name>
<value>yarn</value>
</property>
<property>
<name>yarn.apps.stagingDir</name> <value>/user/%USERNAME%/staging</value>
</property>
<property>
<name>mapreduce.jobtracker.address</name>
<value>local</value>
</property>
</configuration>

Check if C:\BigData\hadoop-3.1.2\etc\hadoop\slaves file is present, if it’s not then created one and add localhost in it and save it.

Step 10: Format Name Node

To organize the Name Node, open another Windows Command Prompt and run the beneath order. It might give you a few admonitions, disregard them.

- hadoop namenode -format

```
18/08/05 22:43:23 INFO namenode.FSDirectory: ACLs enabled? false
18/08/05 22:43:23 INFO namenode.FSDirectory: XAttrs enabled? true
18/08/05 22:43:24 INFO namenode.NameNode: Caching file names occurring more than 10 times
18/08/05 22:43:24 INFO snapshot.SnapshotManager: Loaded config captureOpenFiles: falseskipCaptureAccessTimeOnlyChange: false
18/08/05 22:43:24 INFO util.GSet: Computing capacity for map cachedBlocks
18/08/05 22:43:24 INFO util.GSet: VM type       = 64-bit
18/08/05 22:43:24 INFO util.GSet: 0.25% max memory 889 MB = 2.2 MB
18/08/05 22:43:24 INFO util.GSet: capacity      = 2^18 = 262144 entries
18/08/05 22:43:24 INFO metrics.TopMetrics: NNTop conf: dfs.namenode.top.window.num.buckets = 10
18/08/05 22:43:24 INFO metrics.TopMetrics: NNTop conf: dfs.namenode.top.num.users = 10
18/08/05 22:43:24 INFO metrics.TopMetrics: NNTop conf: dfs.namenode.top.windows.minutes = 1,5,25
18/08/05 22:43:24 INFO namenode.FSNamesystem: Retry cache on namenode is enabled
18/08/05 22:43:24 INFO namenode.FSNamesystem: Retry cache will use 0.03 of total heap and retry cache entry expiry time is 600000 millis
18/08/05 22:43:24 INFO util.GSet: Computing capacity for map NameNodeRetryCache
18/08/05 22:43:24 INFO util.GSet: VM type       = 64-bit
18/08/05 22:43:24 INFO util.GSet: 0.029999999329447746% max memory 889 MB = 273.1 KB
18/08/05 22:43:24 INFO util.GSet: capacity      = 2^15 = 32768 entries
Re-format filesystem in Storage Directory C:\BigData\hadoop-2.9.1\data\namenode ? (Y or N) Y
18/08/05 22:43:30 INFO namenode.FSImage: Allocated new BlockPoolId: BP-966647300-10.0.0.169-1533523410205
18/08/05 22:43:30 INFO common.Storage: Will remove files: [C:\BigData\hadoop-2.9.1\data\namenode\current\edits_inprogress_0000000000000000
00000, C:\BigData\hadoop-2.9.1\data\namenode\current\fsimage_0000000000000000000.md5, C:\BigData\hadoop-2.9.1\data\namenode\current\seen_t
18/08/05 22:43:30 INFO common.Storage: Storage directory C:\BigData\hadoop-2.9.1\data\namenode has been successfully formatted.
18/08/05 22:43:30 INFO namenode.FSImageFormatProtobuf: Saving image file C:\BigData\hadoop-2.9.1\data\namenode\current\fsimage.ckpt_000000
18/08/05 22:43:30 INFO namenode.FSImageFormatProtobuf: Image file C:\BigData\hadoop-2.9.1\data\namenode\current\fsimage.ckpt_0000000000000
18/08/05 22:43:30 INFO namenode.NNStorageRetentionManager: Going to retain 1 images with txid >= 0
18/08/05 22:43:30 INFO namenode.NameNode: SHUTDOWN_MSG:
/************************************************************
SHUTDOWN_MSG: Shutting down NameNode at [redacted]
************************************************************/

C:\Users\User>
```

Step 11: Launch Hadoop

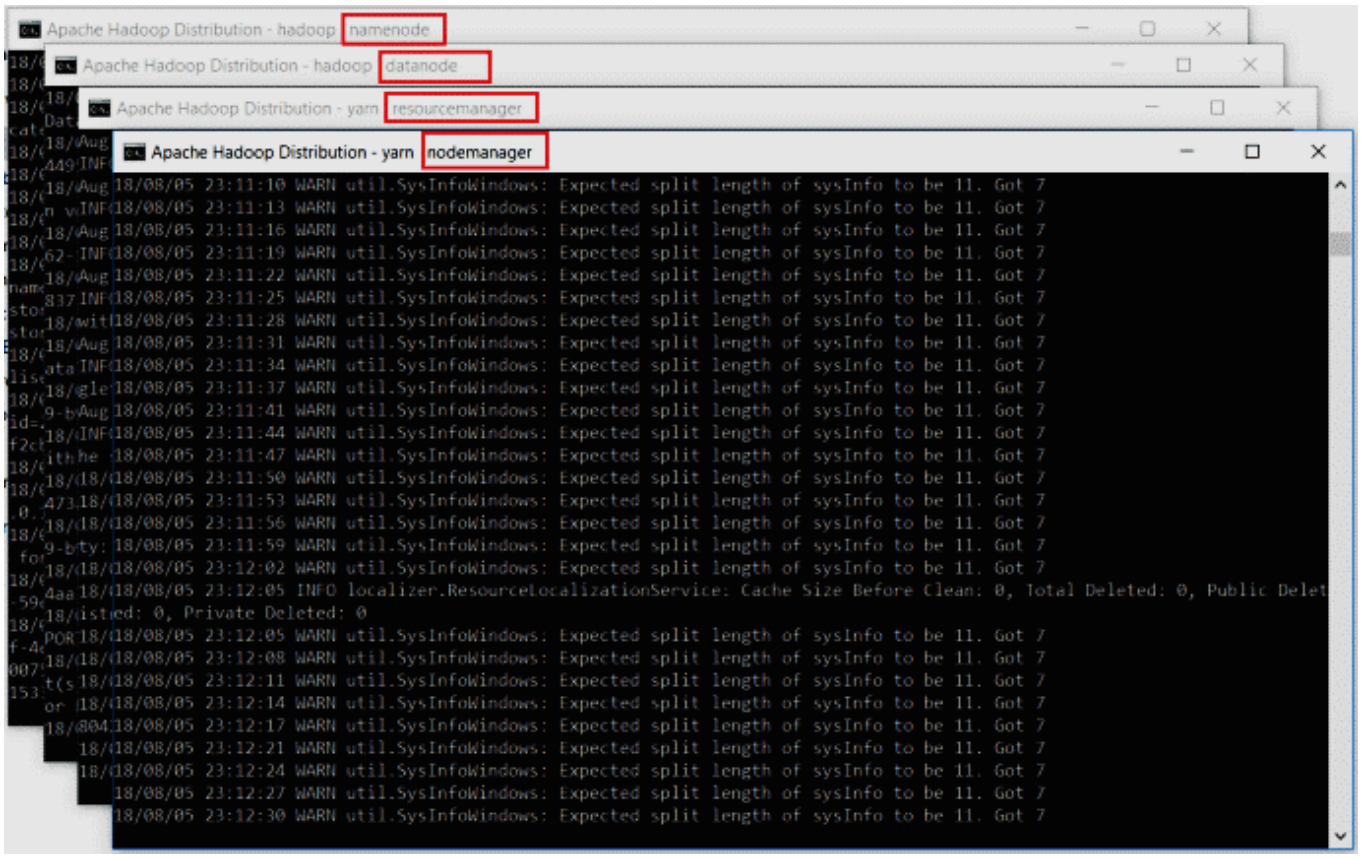
Open another Windows Command brief, make a point to run it as an Administrator to maintain a strategic distance from authorization mistakes. When opened, execute the beginning all.cmd order. Since we have added %HADOOP_HOME%\sbin to the PATH variable, you can run this order from any envelope. In the event that you haven't done as such, at that point go to the %HADOOP_HOME%\sbin organizer and run the order.

```
C:\Users\User>
C:\Users\User>start-all.cmd
This script is Deprecated. Instead use start-dfs.cmd and start-yarn.cmd
starting yarn daemons

C:\Users\User>
```

You can check the given below screenshot for your reference 4 new windows will open and cmd terminals for 4 daemon processes like as follows.

- namenode
- datanode
- node manager
- resource manager

Don't close these windows, minimize them. Closing the windows will terminate the daemons. You can run them in the background if you don't like to see these windows.

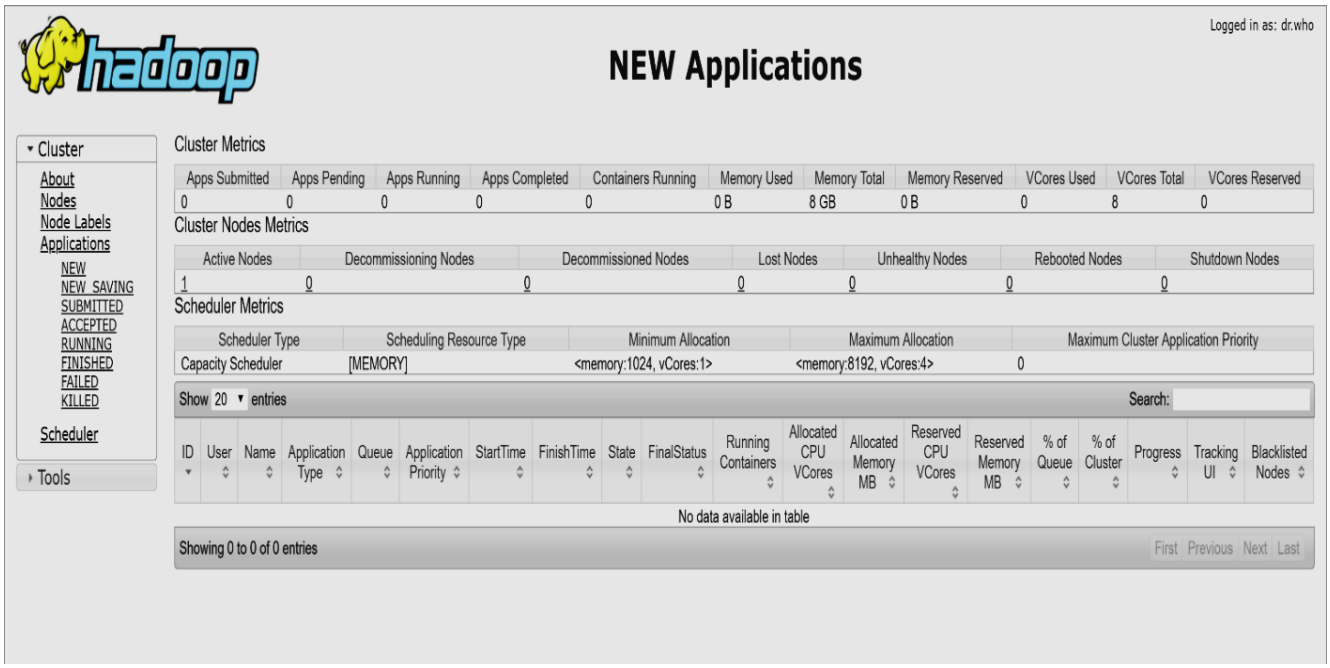


Step 12: Hadoop Web UI

In conclusion, how about we screen to perceive how are Hadoop daemons are getting along. Also you can utilize the Web UI for a wide range of authoritative and observing purposes. Open your program and begin.

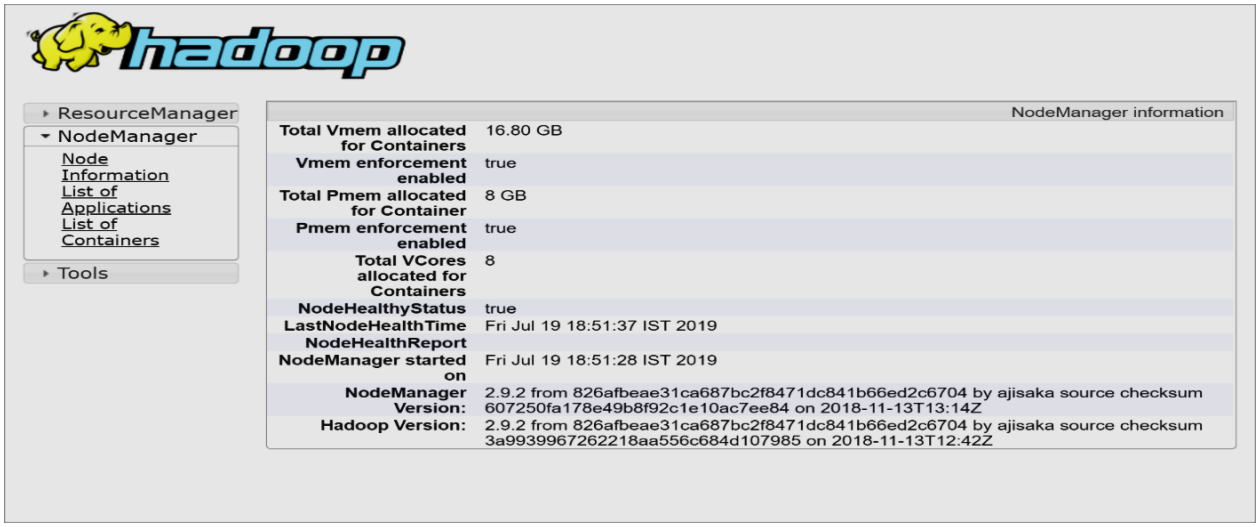
Step 13: Resource Manager

Open localhost:8088 to open Resource Manager



Step 14: Node Manager

Open localhost:8042 to open Node Manager



Step 15: Name Node

Open localhost:9870 to check out the health of Name Node

- ResourceManager
- NodeManager
  - Node Information
  - List of Applications
  - List of Containers
- Tools

NodeManager information	
Total Vmem allocated for Containers	16.80 GB
Vmem enforcement enabled	true
Total Pmem allocated for Container	8 GB
Pmem enforcement enabled	true
Total VCores allocated for Containers	8
NodeHealthyStatus	true
LastNodeHealthTime	Fri Jul 19 18:51:37 IST 2019
NodeHealthReport	
NodeManager started on	Fri Jul 19 18:51:28 IST 2019
NodeManager Version:	2.9.2 from 826afbeae31ca687bc2f8471dc841b66ed2c6704 by ajisaka source checksum 607250fa178e49b8f92c1e10ac7ee84 on 2018-11-13T13:14Z
Hadoop Version:	2.9.2 from 826afbeae31ca687bc2f8471dc841b66ed2c6704 by ajisaka source checksum 3a9939967262218aa556c684d107985 on 2018-11-13T12:42Z

Step 16: Data Node

Open localhost:9864 to check out Data Node

Hadoop

Overview | Datanodes | Datanode Volume Failures | Snapshot | Startup Progress | Utilities

Overview '0.0.0.0:19000' (active)

Started:	Fri Jul 19 18:51:36 +0530 2019
Version:	2.9.2, r826afbeae31ca687bc2f8471dc841b66ed2c6704
Compiled:	Tue Nov 13 18:12:00 +0530 2018 by ajisaka from branch-2.9.2
Cluster ID:	CID-2d1ae9ff-db10-4918-84e8-f5135c6aaef6
Block Pool ID:	BP-620966873-192.168.43.198-1563542402831

Summary

Security is off.

Safemode is off.

4 files and directories, 1 blocks = 5 total filesystem object(s).

Heap Memory used 128.37 MB of 164 MB Heap Memory. Max Heap Memory is 889 MB.

Non Heap Memory used 46.54 MB of 47.19 MB Commited Non Heap Memory. Max Non Heap Memory is <unbounded>.

Configured Capacity:	164.98 GB
DFS Used:	353 B (0%)
Non DFS Used:	20.7 GB
DFS Remaining:	144.28 GB (87.46%)
Block Pool Used:	353 B (0%)
DataNodes usages% (Min/Median/Max/stdDev):	0.00% / 0.00% / 0.00% / 0.00%
Live Nodes	1 (Decommissioned: 0, In Maintenance: 0)
Dead Nodes	0 (Decommissioned: 0, In Maintenance: 0)
Decommissioning Nodes	0
Entering Maintenance Nodes	0
Total Datanode Volume Failures	0 (0 B)
Number of Under-Replicated Blocks	0
Number of Blocks Pending Deletion	0
Block Deletion Start Time	Fri Jul 19 18:51:36 +0530 2019
Last Checkpoint Time	Fri Jul 19 18:50:03 +0530 2019

NameNode Journal Status

Current transaction ID: 9

Journal Manager	State
FileJournalManager(root=F:\BIGDATA\hadoop-2.9.2\data\namenode)	EditLogFileOutputStream(F:\BIGDATA\hadoop-2.9.2\data\namenode\current\edits_inprogress_0000000000000000001)

NameNode Storage

Storage Directory	Type	State
F:\BIGDATA\hadoop-2.9.2\data\namenode	IMAGE_AND_EDITS	Active

DFS Storage Types

Storage Type	Configured Capacity	Capacity Used	Capacity Remaining	Block Pool Used	Nodes In Service
DISK	164.98 GB	353 B (0%)	144.28 GB (87.46%)	353 B	1

Hadoop, 2018.

Assignment No. 2

Use Hadoop File System (HDFS) commands to perform basic operations like creating directories, uploading files, listing files, and deleting files in HDFS.

Open ->(Oracle VM VirtualBox) Cloudera->open Terminal->Perform following Commands

#Linux Command

1.Create File in Linux

```
vi firstfile.txt
```

2.Show File Contain in Linux

```
cat firstfile.txt
```

#Hadoop Command

1. Create a Directory in HDFS

```
hdfs dfs –mkdir firstdir
```

2. List Files in a Directory in HDFS

```
hdfs dfs –ls
```

3. Upload Files from Local File System to HDFS

Use the `-put` command to upload files from your local file system to HDFS.

```
hdfs dfs –put firstfile.txt /user/cloudera/firstdir
```

4. List Files in a Directory

Use the `-ls` command to list the files in an HDFS directory.

```
hdfs dfs –ls  /user/cloudera/firstdir/
```

5. Display the Contents of a File

Use the `-cat` command to display the contents of a file in HDFS.

```
hdfs dfs -cat  /user/cloudera/firstdir/firstfile.txt
```

6. Delete a File in HDFS

Use the `-rm` command to remove a file from HDFS.

```
hdfs dfs -rm  /user/cloudera/firstdir/firstfile1.txt
```

Assignment No. 3

Open ->(Oracle VM VirtualBox) Cloudera->Click on java eclipse->Click on file menu ->Create Java Project-> Add Class and write java code

Implement a Java program to interact with HDFS (reading and writing files).

```
import java.io.File;
public  class filehand  {
  public static void main(String []args) throws IOException
  {
     File obj1=new File("/home/cloudera/Desktop/Firstfile.txt");
     if (obj1.createNewFile())
        System.out.print("File is Created");
     else
        System.out.print("File is Already exist");
     FileWrite w1=new FileWriter("/home/cloudera/Desktop/Firstfile.txt");
     w1.write("Welcome my first file is write");
     w1.close();
     Scanner r1=new Scanner(obj1);
     while(r1.hasNextLine())
     {
          String data=r1.nextLine();
          System.out.println(data);
     }
   }
}
```

Assignment No. 4

Open ->(Oracle VM VirtualBox) Cloudera->open Terminal->Perform following Commands

Use Hadoop's built-in commands to manage files and directories.

1. Create Directories in HDFS.

```
hdfs dfs –mkdir firstdir
```

2. Upload Files from Local File System to HDFS

Use the `-put` command to upload files from your local file system to HDFS.

```
hdfs dfs –put firstfile.txt /user/cloudera/firstdir
```

3. List Files in a Directory

Use the `-ls` command to list the files in an HDFS directory.

```
hdfs dfs –ls  /user/cloudera/firstdir/
```

5. Display the Contents of a File

Use the `-cat` command to display the contents of a file in HDFS.

```
hdfs dfs -cat  /user/cloudera/firstfile.txt
```

6. Copy Files from HDFS to Local File System

Use the `-get` command to copy files from HDFS to your local file system.

```
hdfs dfs -get  /user/cloudera/firstdir/firstfile.txt /home/cloudera/lindir
```

7. Delete a File in HDFS

Use the `-rm` command to remove a file from HDFS.

```
hdfs dfs -rm  /user/cloudera/firstdir/firstfile1.txt
```

8. Delete a Directory in HDFS

Use the `-rm -r` command to delete a directory and its contents from HDFS.

```
hdfs dfs -rm -r /user/cloudera/firstdir  (For Non Empty Directory
hdfs dfs -rmdir  /user/cloudera/firstdir     (For Empty Directory )
```

Assignment No. 5

Step 1-Open ->(Oracle VM VirtualBox) Cloudera->

Step 2-Click on java eclipse->Click on file menu ->Create Java Project-> Go To Libraries-> Add Libraries (click add libraries button-> Go To File System-usr-lib-hadoop folder->add all libraries & again add all libraries under client folder)

Step 3-Right Click on project name->add new class->write the following code

Implement Map Side Join and Reduce Side Join.

(Write hadoop code to implement Map Reduce application count number of word in file)

```
import java.io.IOException;
import java.util.StringTokenizer;
import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Job;
import org.apache.hadoop.mapreduce.Mapper;
import org.apache.hadoop.mapreduce.Reducer;
import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;

public class WordCount {

  public static class TokenizerMapper
       extends Mapper<Object, Text, Text, IntWritable>{

    private final static IntWritable one = new IntWritable(1);
    private Text word = new Text();

    public void map(Object key, Text value, Context context
                    ) throws IOException, InterruptedException {
      StringTokenizer itr = new StringTokenizer(value.toString());
      while (itr.hasMoreTokens()) {
        word.set(itr.nextToken());
        context.write(word, one);
      }
    }
  }
  public static class IntSumReducer
       extends Reducer<Text,IntWritable,Text,IntWritable> {
    private IntWritable result = new IntWritable();

    public void reduce(Text key, Iterable<IntWritable> values,
                       Context context
                       ) throws IOException, InterruptedException {
      int sum = 0;
      for (IntWritable val : values) {
        sum += val.get();
      }
      result.set(sum);
      context.write(key, result);
    }
  }
  public static void main(String[] args) throws Exception {
    Configuration conf = new Configuration();
    Job job = Job.getInstance(conf, "word count");
    job.setJarByClass(WordCount.class);
    job.setMapperClass(TokenizerMapper.class);
    job.setCombinerClass(IntSumReducer.class);
    job.setReducerClass(IntSumReducer.class);
```

```
    job.setOutputKeyClass(Text.class);
    job.setOutputValueClass(IntWritable.class);
    FileInputFormat.addInputPath(job, new Path(args[0]));
    FileOutputFormat.setOutputPath(job, new Path(args[1]));
    System.exit(job.waitForCompletion(true) ? 0 : 1);
  }
}
```

File Link
https://hadoop.apache.org/docs/current/hadoop-mapreduce-client/hadoop-mapreduce-client-core/MapReduceTutorial.html#Source_Code

Step 4: Export Java Eclipse Project Jar File to Cloudera

Step 5. Make firstfile.txt file vi editor ->Write data

Step 6: Perform Below commands on terminal

Command Map Reduce Code

1) Transfer all local file to hadoop

Hdfs dfs –put firstfile.txt /user/cloudera

Hdfs dfs –put WordCount.jar /user/cloudera

2) Run Java Jar File for Map Reduce Operation

hadoop jar WordCount.jar WordCount firstfile.txt outputfile

3) List outputfile

hdfs dfs –ls /user/cloudera/outputfile

4) Show outputfile

hdfs dfs –cat /user/cloudera/output

Assignment No. 6

Implement Secondary Sorting. (Write hadoop code to implement Item Sort Program)

```
       ----------Main class------------------------
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;
import org.apache.hadoop.mapreduce.Job;

public class testdriver {
    public static void main(String[] args) throws Exception {
       if (args.length != 2) {
          System.out.printf("Usage: WordCount <input dir> <output
dir>\n");
          System.exit(-1);
                }
                Job job = new Job();

                job.setJarByClass(testdriver.class);
                job.setJobName("Word Count");
                FileInputFormat.setInputPaths(job, new Path(args[0]));
                FileOutputFormat.setOutputPath(job, new Path(args[1]));

                job.setMapperClass(testmap.class);
                job.setReducerClass(testreduce.class);

                job.setMapOutputKeyClass(IntWritable.class);
                job.setMapOutputValueClass(IntWritable.class);

                job.setOutputKeyClass(IntWritable.class);
                job.setOutputValueClass(IntWritable.class);

                boolean success = job.waitForCompletion(true);
                System.exit(success ? 0 : 1);

        }
}


----------Mapper class------------------------

import java.io.IOException;
import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.LongWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Mapper;

public class testmap extends Mapper<LongWritable, Text, IntWritable,
IntWritable> {
@Override
     public void map(LongWritable key, Text value, Context context)
throws IOException, InterruptedException {
         String line = value.toString();
           String[] tokens = line.split(","); // This is the delimiter
between
           int keypart = Integer.parseInt(tokens[0]);
           int valuePart = Integer.parseInt(tokens[1]);
           context.write(new IntWritable(valuePart), new
IntWritable(keypart));
```

```
        }
}



----------Reducer class------------------------

import java.io.IOException;
import org.apache.hadoop.io.IntWritable;

import org.apache.hadoop.mapreduce.Reducer;

public class testreduce extends Reducer<IntWritable, IntWritable,
IntWritable, IntWritable> {
@Override
        public void reduce(IntWritable key, Iterable<IntWritable>
values,
Context context)  throws IOException, InterruptedException {

        for (IntWritable value : values) {

            context.write(value,key);

           }

}
}
```

Step 1: Export Java Eclipse Project Jar File to Cloudera
Step 2. Make Sort.txt file vi editor ->Write data
Step 3: Perform Below commands on terminal

Command Map Reduce Code

5) Transfer all local file to hadoop

```
Hdfs dfs –put sort.txt /user/cloudera

Hdfs dfs –put Sorting.jar /user/cloudera
```

6) Run Java Jar File for Map Reduce Operation

```
hadoop jar Sorting.jar testdriver sort.txt outputsort
```

7) List outputfile

```
hdfs dfs –ls /user/cloudera/outputsort
```

8) Show outputfile

```
hdfs dfs –cat /user/cloudera/outputsort /part-r-00000
```

Assignment No. 7

Pipeline multiple Map Reduce jobs

Job 1 (WordCount) counts the frequency of each word in the input data, while Job 2 (FilterWords) filters out words with a count greater than 2.

Job 1: Word Count (Word frequency count)

This first job counts the occurrences of each word in the input text files.

```
import java.io.IOException;
import java.util.StringTokenizer;
import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Job;
import org.apache.hadoop.mapreduce.Mapper;
import org.apache.hadoop.mapreduce.Reducer;
import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;

public class WordCount {
    public static class TokenizerMapper extends Mapper<Object, Text, Text,
IntWritable> {
        private final static IntWritable one = new IntWritable(1);
        private Text word = new Text();

        public void map(Object key, Text value, Context context) throws
IOException, InterruptedException {
            String[] words = value.toString().split("\\s+");
            for (String wordStr : words) {
                word.set(wordStr);
                context.write(word, one);
            }
        }
    }

    public static class IntSumReducer extends Reducer<Text, IntWritable,
Text, IntWritable> {
        private IntWritable result = new IntWritable();

        public void reduce(Text key, Iterable<IntWritable> values, Context
context) throws IOException, InterruptedException {
            int sum = 0;
            for (IntWritable val : values) {
                sum += val.get();
            }
            result.set(sum);
            context.write(key, result);
        }
    }
    public static void main(String[] args) throws Exception {
        Configuration conf = new Configuration();
        Job job = Job.getInstance(conf, "word count");
        job.setJarByClass(WordCount.class);
        job.setMapperClass(TokenizerMapper.class);
        job.setCombinerClass(IntSumReducer.class);
        job.setReducerClass(IntSumReducer.class);
        job.setOutputKeyClass(Text.class);
        job.setOutputValueClass(IntWritable.class);
        FileInputFormat.addInputPath(job, new Path(args[0]));
        FileOutputFormat.setOutputPath(job, new Path(args[1]));
        System.exit(job.waitForCompletion(true) ? 0 : 1);
    }
```

```
}
```

Job 2: Filter Words with Frequency Greater Than 2

The second job processes the output of the first job to filter and only output words that have a frequency greater than 2

```
import java.io.IOException;
import java.util.StringTokenizer;
import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Job;
import org.apache.hadoop.mapreduce.Mapper;
import org.apache.hadoop.mapreduce.Reducer;
import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;

public class FilterWords {
    public static class FilterMapper extends Mapper<Object, Text, Text,
IntWritable> {
        private IntWritable count = new IntWritable();

        public void map(Object key, Text value, Context context) throws
IOException, InterruptedException {
            String[] fields = value.toString().split("\t");
            String word = fields[0];
            int wordCount = Integer.parseInt(fields[1]);

            // Output only words with count greater than 2
            if (wordCount > 2) {
                count.set(wordCount);
                context.write(new Text(word), count);
            }
        }
    }

    public static void main(String[] args) throws Exception {
        Configuration conf = new Configuration();
        Job job = Job.getInstance(conf, "filter words");
        job.setJarByClass(FilterWords.class);
        job.setMapperClass(FilterMapper.class);
        job.setOutputKeyClass(Text.class);
        job.setOutputValueClass(IntWritable.class);

        FileInputFormat.addInputPath(job, new Path(args[0]));  // Input
path from the first job's output
        FileOutputFormat.setOutputPath(job, new Path(args[1]));  // Output
path

        System.exit(job.waitForCompletion(true) ? 0 : 1);
    }
}
```

Step 1: Export Java Eclipse Project Jar File to Cloudera
Step 2. Make firstfile.txt file vi editor ->Write data
Step 3: Perform Below commands on terminal

<u>Command Map Reduce Code</u>

9) <u>Transfer all local file to hadoop</u>

```
Hdfs dfs –put firstfile.txt /user/cloudera
Hdfs dfs –put PipLine1.jar /user/cloudera
```

10) <u>Run First job of Java Jar File for Map Reduce Operation</u>

```
hadoop jar PipLine1.jar wordcount firstfile.txt outpip1
```

11) Run Second job of Java Jar File for Map Reduce Operation

```
hadoop jar PipLine1.jar FilterWords outpip1 outpip2
```

12) List outputfile

```
hdfs dfs –ls /user/cloudera/outpip2
```

13) Show outputfile

```
hdfs dfs –cat /user/cloudera/outpip2/part-r-00000
```

Assignment No. 8

Create and use UDFs in Pig Latin scripts (Write hadoop code to convert username in uppercase)

Step 1: Open Java Eclipse -> Make New Project->(PigAss7) Add External Libarary->pig , hadoop ,hadoop 0.20_map_reduce->finish

Step 2: Add class in project -> PigUDF -> Write below code (Create the UDF)

```
import  org.apache.pig.EvalFunc;
import org.apache.pig.data.Tuple;
public class PigUDF extends EvalFunc<String> {
public String exec(Tuple tuple)throws IOException {
   if(tuple ==null) {
        return null;
   }
   String user=(String)tuple.get(0);
   String city=(String)tuple.get(1);
   int score=(Integer)tuple.get(2);
   return user+”,”+city.toUpperCase()+”,”+score;
 }
}
```

Step 3: Export java project in to jar file ->PigAss7.jar

Step 4: Open terminal create cust_us.txt file using following command

vi cust_us

Step 5: Open terminal to create pig file using following command.

vi toupper.pig

Write following code in file

```
REGISTER /home/cloudera/PigAss7.jar
DEFINE toupper PigUDF();
usa = LOAD  ‘/home/cloudera/cust_us.txt’ Using PigStorage(‘,’)
AS (user:chararray,city:chararray,score:int);
usa_upper = FOREACH usa GENERATE toupper(user,city,score);

DUMP usa_upper;
```

Step 6: Run Pig script

```
pig –x  local toupper.pig
```

Assignment No. 9

Integrate UDFs to enhance the functionality of Pig scripts. (Write hadoop code to concatenation two string)

Step 1: Open Java Eclipse -> Make New Project->(PigAss7) Add External Libarary->pig , hadoop ,hadoop 0.20_map_reduce->finish

Step 2: Add class in project -> (Ass8Demo) -> Write below code (Create the UDF)

```
import org.apache.pig.EvalFunc;
import org.apache.pig.data.Tuple;
import org.apache.pig.data.DataByteArray;

public class ConcatStrings extends EvalFunc<DataByteArray> {
    @Override
    public DataByteArray exec(Tuple input) {
        if (input == null || input.size() < 2)
            return null;
        try {
            String str1 = (String) input.get(0);
            String str2 = (String) input.get(1);
            return new DataByteArray((str1 + str2).getBytes());
        } catch (Exception e) {
            return null;
        }
    }
}
```

Step 3: Export java project in to jar file ->Ass8Demo.jar

Step 4: Open terminal create strdata.txt file using following command

vi strdata.txt

hi,hello
good,morning
welcome,RCPET's IMRD

Step 5: Open terminal to create pig file using following command.

vi strcat.pig

Write following code in file

```
REGISTER /home/cloudera/Ass8Demo.jar
DEFINE ConcatStrings ConcatStrings();
data = LOAD  '/home/cloudera/strdata.txt' Using
PigStorage(',') AS (str1:chararray,str2:chararray);
result = FOREACH data GENERATE ConcatStrings(str1,str2);
DUMP result;
```

Step 6: Run Pig script

```
pig -x  local strcat.pig
```

Assignment No 10

Implement and execute HiveQL queries to perform data retrieval and manipulation.

1. **Setting up Hive**
   Hive

2. **Creating a Hive Table**
```
CREATE TABLE employees (
   emp_id INT,
   emp_name STRING,
   emp_salary DOUBLE,
   emp_department STRING
) ROW FORMAT DELIMITED
FIELDS TERMINATED BY ','
STORED AS TEXTFILE;
```

3. **Data Manipulation**
   a. **Inserting Data into the Table:**
```
INSERT INTO TABLE employees VALUES (1001, 'John Doe', 60000.00,'Engineering');
```
   b. **Updating Data:**
```
INSERT OVERWRITE TABLE employees
SELECT emp_id, emp_name, CASE
                WHEN emp_salary < 50000 THEN emp_salary * 1.1
                ELSE emp_salary
              END AS emp_salary,
              emp_department
FROM employees;
```
   c. **Deleting Data:**
```
INSERT OVERWRITE TABLE employees
SELECT * FROM employees WHERE emp_id != 1001;
```

4. **Basic Data Retrieval Queries**
   a. **Select all records:**
```
SELECT * FROM employees;
```
   b. **Select specific columns:**
```
SELECT emp_name, emp_salary FROM employees;
```
   c. **Select with a WHERE clause:**
```
SELECT * FROM employees WHERE emp_salary > 50000;
```

Assignment No 11

Perform operations like joins, group by, and aggregations in Hive.

1. **Creating a Hive Table**

```
CREATE TABLE employees (
   emp_id INT,
   emp_name STRING,
   emp_salary DOUBLE,
   emp_department STRING
) ROW FORMAT DELIMITED
FIELDS TERMINATED BY ','
STORED AS TEXTFILE;

CREATE TABLE departments (
   dept_id INT,
   dept_name STRING,
) ROW FORMAT DELIMITED
FIELDS TERMINATED BY ','
STORED AS TEXTFILE;
```

Insert 5 record on each table by using insert command

2. **Joins**

```
SELECT e.emp_name, d.dept_name
FROM employees e
JOIN departments d
ON e.emp_department = d.dept_id;
```

3. **Aggregate Functions**

a. **Count the number of employees:**

```
SELECT COUNT(*) FROM employees;
```

b. **Find the average salary:**

```
SELECT AVG(emp_salary) FROM employees;
```

c. **Find the highest salary:**

```
SELECT MAX(emp_salary) FROM employees;
```

4. **Group By**

Group by department and get average salary per department:

```
SELECT emp_department, AVG(emp_salary) FROM employees GROUP BY
emp_department;
```